

Examen Parcial

Matemática Finita II

Grado en Ingeniería de Datos e Inteligencia Artificial
2025

Dr. Miguel Angel Navarro Burgos

Nombre completo: _____
Código identificador: 11
Nota: _____

Instrucciones: Resuelva cada pregunta de manera ordenada, exponiendo con claridad los procedimientos utilizados y justificando rigurosamente sus respuestas. Además, resalte el resultado obtenido.

Duración estimada: 120 minutos

1. (2.5 puntos) Resuelva la siguiente recurrencia no lineal de grado 3:

$$T(n) = 8T\left(\frac{n}{2}\right) + 12n^3T\left(\frac{n}{2}\right)^{2/3} + 6n^6T\left(\frac{n}{2}\right)^{1/3} + n^9, \quad T(1) = 1$$

- (a) (1 punto) Realice un cambio de variable adecuado para linealizar la recurrencia.
(b) (1 punto) Resuelva la recurrencia obtenida.
(c) (0.5 puntos) Determine el orden exacto de complejidad $\Theta(\cdot)$ de $T(n)$.

Pista: Observe que la expresión del segundo miembro recuerda al desarrollo de $(a+b)^3$.

2. (2.5 puntos) Considere la siguiente recurrencia:

$$T(n) = aT\left(\frac{n}{b}\right) + n^p \left(\frac{\alpha n^q + \phi(n)}{\beta n^r + \psi(n)} \right) \log^k n, \quad T(1) = 1$$

donde $a, b, \alpha, \beta > 0$, $p, q, r, k \in \mathbb{N}$, y las funciones $\phi(n), \psi(n)$ están acotadas, es decir

$$|\phi(n)| \leq C_1, \quad |\psi(n)| \leq C_2.$$

- (a) (1 punto) Demuestre que

$$f(n) = n^p \left(\frac{\alpha n^q + \phi(n)}{\beta n^r + \psi(n)} \right) \log^k n = \Theta(n^{p+q-r} \log^k n).$$

- (b) (0.5 puntos) Calcule $n^{\log_b a}$.
 (c) (0.5 puntos) Determine qué caso del teorema maestro se aplica dependiendo de la relación entre $p + q - r$ y $\log_b a$.
 (d) (0.5 puntos) Escriba la complejidad asintótica de $T(n)$.

Pista: Use que si $\phi(n)$ y $\psi(n)$ están acotadas, entonces el comportamiento dominante del cociente lo determinan los términos polinómicos.

3. (2.5 puntos) Resuelva la siguiente recurrencia lineal no homogénea:

$$T(n) - 4T(n-1) + 4T(n-2) = 2^n n, \quad T(0) = 1, \quad T(1) = 4$$

- (a) (0.75 puntos) Resuelva la ecuación homogénea asociada.
 (b) (1 punto) Determine una solución particular adecuada para la recurrencia completa.
 (c) (0.75 puntos) Obtenga la expresión cerrada de $T(n)$ usando las condiciones iniciales.

Pista: El segundo miembro es de la forma $2^n p(n)$, con $p(n)$ polinomio de grado 1. Compruebe si 2 es raíz de la ecuación característica y con qué multiplicidad.

4. (2.5 puntos) Resuelva la siguiente recurrencia lineal no homogénea:

$$T(n) - 5T(n-1) + 6T(n-2) = 2^n + 3^n n, \quad T(0) = 2, \quad T(1) = 7$$

- (a) (0.75 puntos) Resuelva la ecuación homogénea asociada.
 (b) (1 punto) Determine una solución particular para cada uno de los términos del segundo miembro.
 (c) (0.75 puntos) Obtenga la expresión cerrada de $T(n)$.

Pista: Trate por separado los términos 2^n y $3^n n$. Compruebe si 2 o 3 son raíces de la ecuación característica.

5. (5 puntos) Se desea determinar si un número entero $n \geq 2$ es primo.

Considere los siguientes dos enfoques:

- **Algoritmo 1:** comprobar si n tiene divisores entre 2 y $n - 1$.
 - **Algoritmo 2:** comprobar si n tiene divisores entre 2 y $\lfloor \sqrt{n} \rfloor$.
- (a) (1.5 puntos) Escriba en pseudocódigo el algoritmo 1 y analice su complejidad temporal y espacial.
- (b) (1.5 puntos) Escriba en pseudocódigo el algoritmo 2 y analice su complejidad temporal y espacial.
- (c) (1 punto) Compare ambos algoritmos desde el punto de vista teórico y práctico.
- (d) (1 punto) Determine cuál sería más adecuado para valores grandes de n , justificando la respuesta.

Pista: Si n es compuesto y $n = ab$, entonces al menos uno de los factores cumple $a \leq \sqrt{n}$.

6. (5 puntos) Se desea determinar si un número entero positivo n es primo.

Se proponen cuatro estrategias:

Algoritmo 3 Primo versión A

```

1: function PRIMOA( $n$ )
2:   for  $d = 2$  to  $n - 1$  do
3:     if  $n \bmod d = 0$  then
4:       return Falso
5:     end if
6:   end for
7:   return Verdadero
8: end function
```

Algoritmo 4 Primo versión B

```

1: function PRIMOB( $n$ )
2:   for  $d = 2$  to  $\lfloor n/2 \rfloor$  do
3:     if  $n \bmod d = 0$  then
4:       return Falso
5:     end if
6:   end for
7:   return Verdadero
8: end function
```

Algoritmo 5 Primo versión C

```

1: function PRIMOC( $n$ )
2:   for  $d = 2$  to  $\lfloor \sqrt{n} \rfloor$  do
3:     if  $n \bmod d = 0$  then
4:       return Falso
5:     end if
6:   end for
7:   return Verdadero
8: end function

```

Algoritmo 6 Primo versión D

```

1: function PRIMOD( $n$ )
2:   if  $n < 2$  then
3:     return Falso
4:   end if
5:   if  $n = 2$  then
6:     return Verdadero
7:   end if
8:   if  $n \bmod 2 = 0$  then
9:     return Falso
10:  end if
11:  for  $d = 3$  to  $\lfloor \sqrt{n} \rfloor$  step 2 do
12:    if  $n \bmod d = 0$  then
13:      return Falso
14:    end if
15:  end for
16:  return Verdadero
17: end function

```

Para estimar la complejidad suponga que n es primo.

- Determine la complejidad temporal de cada algoritmo en notación Θ (3 pts).
- Compare los algoritmos y determine cuál es el más eficiente (1 pt).
- Explique por qué no es necesario comprobar divisores mayores que $\sqrt{n}$ (1 pt).

Pista: Si $n = ab$ y $a > \sqrt{n}$, entonces necesariamente $b < \sqrt{n}$.

- (5 puntos) Se desea determinar si un elemento x pertenece a un array ordenado A de tamaño n .

Se proponen tres estrategias:

Algoritmo 7 Búsqueda A

```

1: function BUSCARA( $A, x$ )
2:   for  $i = 1$  to  $n$  do
3:     if  $A[i] = x$  then
4:       return Verdadero
5:     end if
6:   end for
7:   return Falso
8: end function

```

Algoritmo 8 Búsqueda C

```

1: function BUSCARC( $A, x$ )
2:    $i \leftarrow 1$ 
3:   while  $i \leq n$  and  $A[i] < x$  do
4:      $i \leftarrow 2i$ 
5:   end while
6:   Buscar  $x$  en el intervalo
7:    $[i/2, i]$  usando búsqueda binaria
8: end function

```

Algoritmo 9 Búsqueda B

```

1: function BUSCARB( $A, x$ )
2:    $l \leftarrow 1$ 
3:    $r \leftarrow n$ 
4:   while  $l \leq r$  do
5:      $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
6:     if  $A[m] = x$  then
7:       return Verdadero
8:     else if  $A[m] < x$  then
9:        $l \leftarrow m + 1$ 
10:    else
11:       $r \leftarrow m - 1$ 
12:    end if
13:  end while
14:  return Falso
15: end function

```

Suponga que el array está ordenado y que x no pertenece al array.

- Determine la complejidad temporal de cada algoritmo en notación Θ (3 pts).
- Compare los tres algoritmos y determine cuál es el más eficiente para valores grandes de n (1 pt).
- Explique en qué situaciones el algoritmo C puede ser preferible al algoritmo B (1 pt).

Pista: En el algoritmo C el tamaño del intervalo crece exponencialmente.

- (5 puntos) Dado un array $A[1..n]$ de números enteros, se desea calcular la suma máxima de un subarray contiguo.

Se proponen cuatro estrategias:

Algoritmo 10 Subarray versión A

```

1: function MAXA( $A$ )
2:    $best \leftarrow -\infty$ 
3:   for  $i = 1$  to  $n$  do
4:     for  $j = i$  to  $n$  do
5:       calcular suma de  $A[i..j]$ 
6:       actualizar  $best$ 
7:     end for
8:   end for
9:   return  $best$ 
10: end function

```

Algoritmo 11 Subarray versión B

```

1: function MAXB( $A$ )
2:    $best \leftarrow -\infty$ 
3:   for  $i = 1$  to  $n$  do
4:      $sum \leftarrow 0$ 
5:     for  $j = i$  to  $n$  do
6:        $sum \leftarrow sum + A[j]$ 
7:       actualizar  $best$ 
8:     end for
9:   end for
10:  return  $best$ 
11: end function

```

Algoritmo 12 Subarray versión C

```

1: function MAXC( $A, l, r$ )
2:   if  $l = r$  then
3:     return  $A[l]$ 
4:   end if
5:    $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
6:   calcular mejor subarray izquier-
     da
7:   calcular mejor subarray derecha
8:   calcular mejor cruzando el medio
9:   return máximo de los tres
10: end function

```

Algoritmo 13 Subarray versión D

```

1: function MAXD( $A$ )
2:    $best \leftarrow A[1]$ 
3:    $current \leftarrow A[1]$ 
4:   for  $i = 2$  to  $n$  do
5:      $current \leftarrow$ 
        $\max(A[i], current + A[i])$ 
6:      $best \leftarrow \max(best, current)$ 
7:   end for
8:   return  $best$ 
9: end function

```

- (a) Determine la complejidad temporal de cada algoritmo en notación Θ (3 pts).
- (b) Compare los algoritmos e indique cuál es el más eficiente (1 pt).
- (c) Escriba la recurrencia asociada al algoritmo C y resuélvala mediante el teorema maestro (1 pt).

Pista: El algoritmo C divide el problema en dos subproblemas de tamaño $n/2$.